

DATE : 15.09.2026

DT/NT : DT

LESSON : GIT & GITHUB

SUBJECT: Git & Github-1

BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Contents:

- Why Do We Need Version Control?
- What is Git?
- Git Installation
- Git Commands



Why Do We Need Version Control?

WITHOUT VERSION CONTROL

project-final.zip

project-final-v2.zip

project-final-v2-fixed.zip

project-FINAL-final.zip

Which one is really the latest?

Git

WITH VERSION CONTROL



Every change becomes a clear, traceable version.

- ✓ **Track changes over time** — See exactly what changed.
- ✓ **Restore earlier versions** — Recover quickly from mistakes.
- ✓ **Work safely as a team** — Multiple developers can collaborate.
- ✓ **Know who changed what** — Commits keep author and history.

Version control gives us a reliable history instead of a folder full of confusing “final” files.

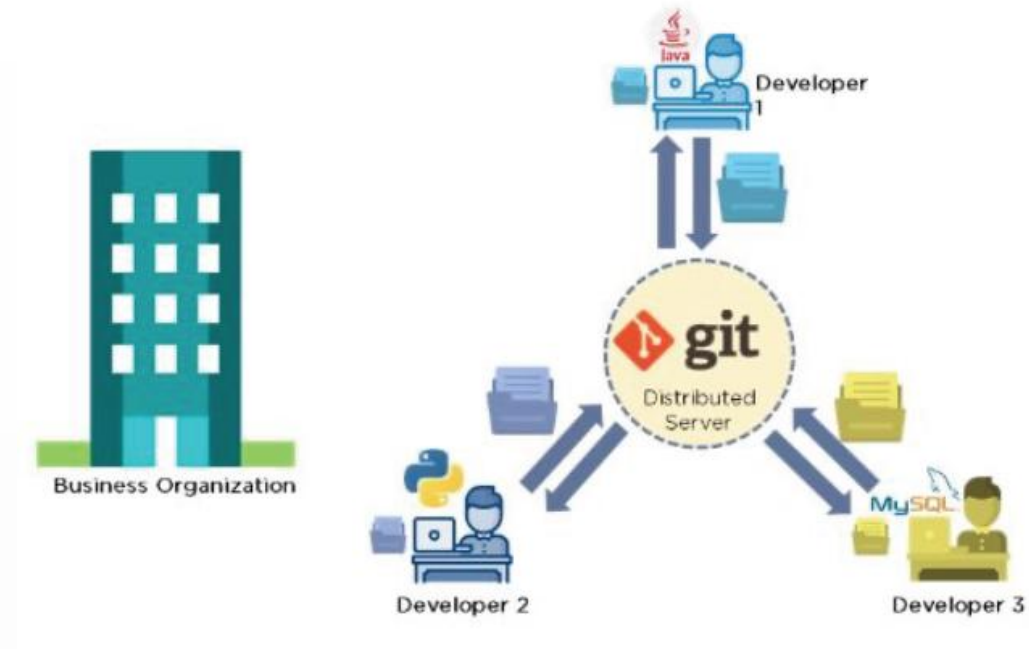
What is Git?

- ❖ Git is a "version control system" that records changes.
- ❖ Git has a distributed structure, meaning each user has their own local copy. This allows you to work offline and save changes locally.



What is Git?

- ❖ Git can be used through command-line tools, IDEs and graphical interfaces. In DevOps, we will primarily learn the command line.
 - ❖ Once you learn the basic commands and shortcuts, you can effectively manage your projects' version control as Git repositories (repository or repo).
- ❖ Git can manage different versions and changes of your project files through branches (branch)..
- ❖ Each change is called a "commit" and is identified by a unique identifier (hash)..



Why Use Git and GitHub?



Version Control System

- Managing versions locally
- Ability to revert errors
- Runs locally

Tracks:

- files
- changes
- commits
- branches



Cloud platform that hosts Git repositories

- Adds:
 - collaboration
 - pull requests
 - issues
 - code review
 - automation

Git Installation



Download Link
<https://git-scm.com/>

Checking installed version of Git in terminal
`git --version || git -v`

Git Config

Configuring Username and Email Address

```
git config --global user.name "John Doe"
```

```
git config --global user.email "john.doe@example.com"
```

To check the configurations, write the commands without values.

Configuring the default branch

```
git config --global init.defaultBranch main
```

Configuring Editor Setting (optional)

```
git config --global core.editor "nano"
```

Enabling Colorful Outputs

```
git config --global color.ui true
```

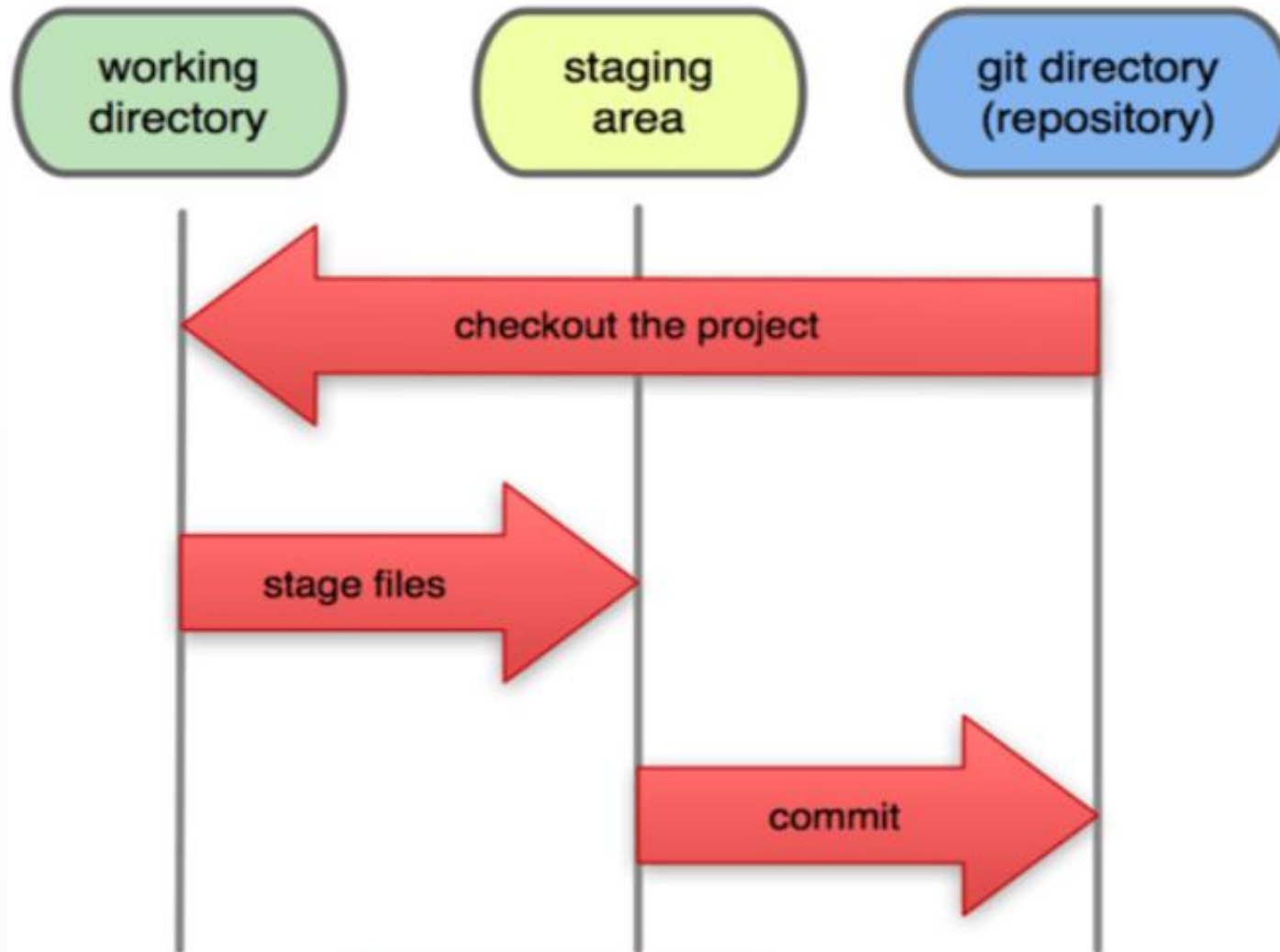


Git Config

```
git config --global
```

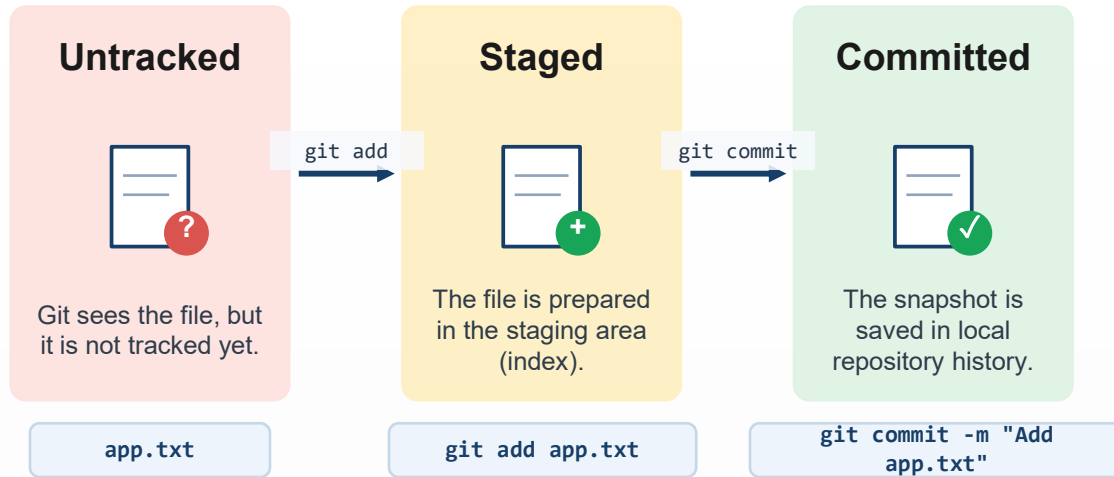
Git Working Structure

Local Operations

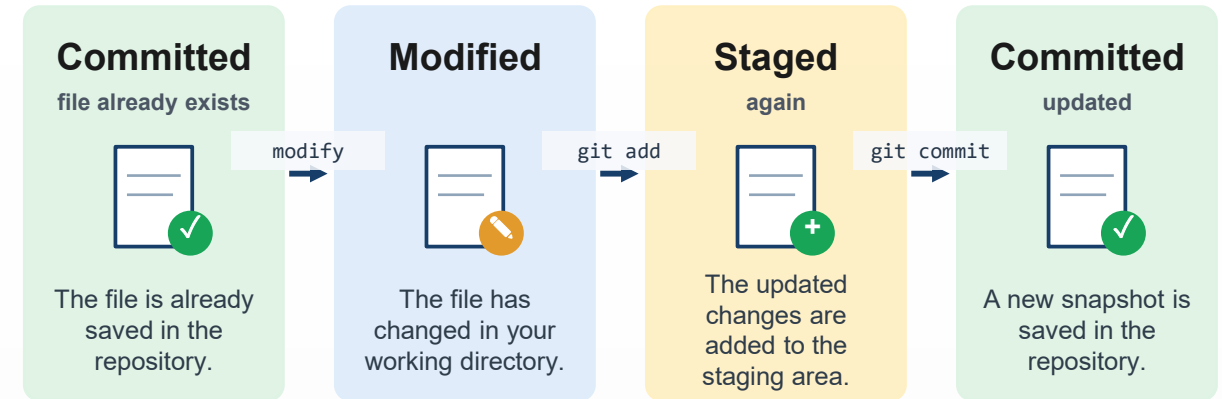


Git File States

1. New file (from scratch)



2. After modifying a committed file



3. See the current state with `git status`

```
$ git status
On branch main
```

```
Changes to be committed:
  new file:   app.txt
```

```
Changes not staged for commit:
  modified:   config.txt
```

```
Untracked files:
  notes.txt
```

What this means

- **app.txt → staged**
Ready for the next commit.
- **config.txt → modified**
Changed, but not staged yet.
- **notes.txt → untracked**
Git has not started tracking it.

Key Takeaways

- ✓ **Untracked → Staged → Committed** — Files move through clear states.
- ✓ **Modified** — A committed file becomes Modified after you edit it.
- ✓ **git status** — Shows the current state of your files.
- ✓ **git add** — Moves changes into the staging area.
- ✓ **git commit** — Saves a staged snapshot into repository history.

Creating a Git Project

Initialize a Directory as a Git Repository

```
git init
```

To check, you can type "ls -la" in the terminal..

Create a file and add it to the repository

```
echo "Hello, Git!" > myFile.txt
```

Check that the file is in the staging area.

```
git status
```

```
git add myFile.txt
```

Check that the file is in the staging area.

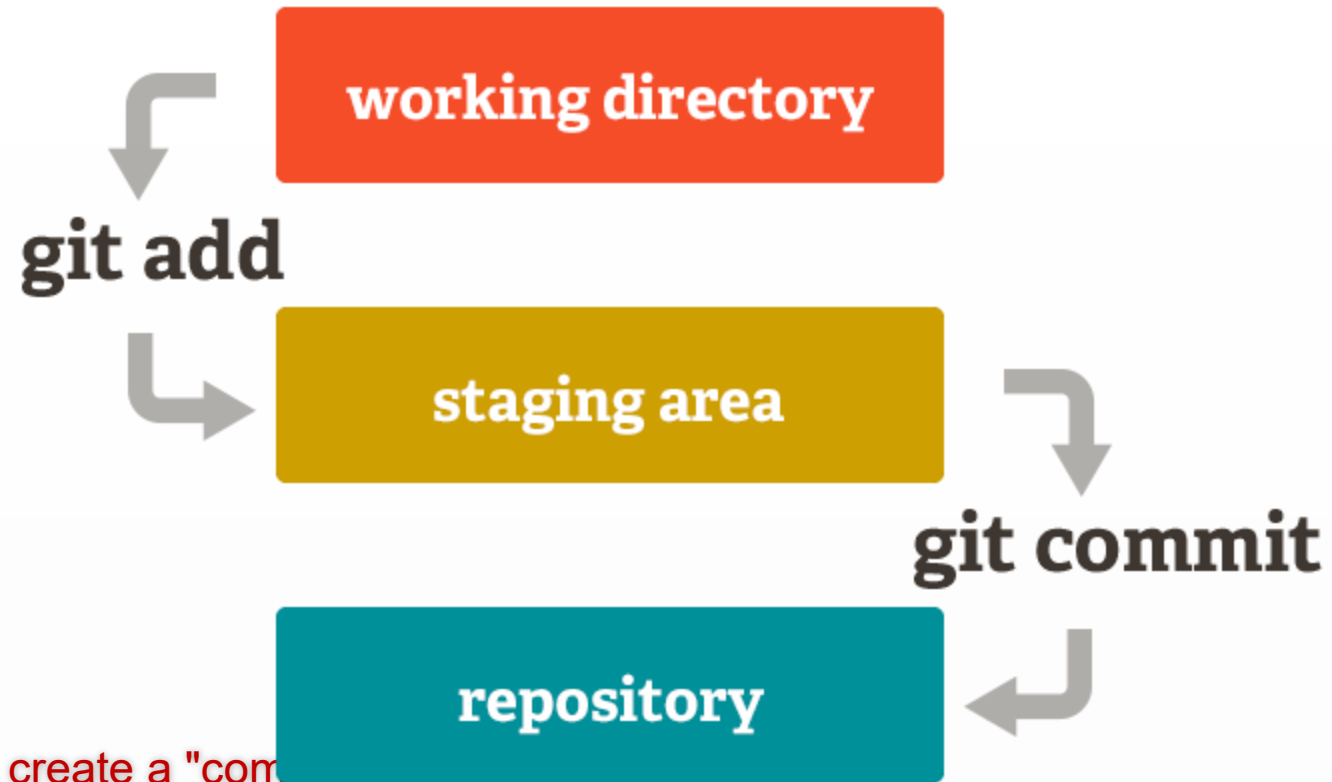
```
git status
```

Having added the file to the repository, now let's create a "commit"

```
git commit -m "first commit"
```

Let's view the project we created

```
git log
```



Git Diff

View All Changes in the Working Area

`git diff`

Working Directory



Staging Area

`git diff --staged`

Staging Area



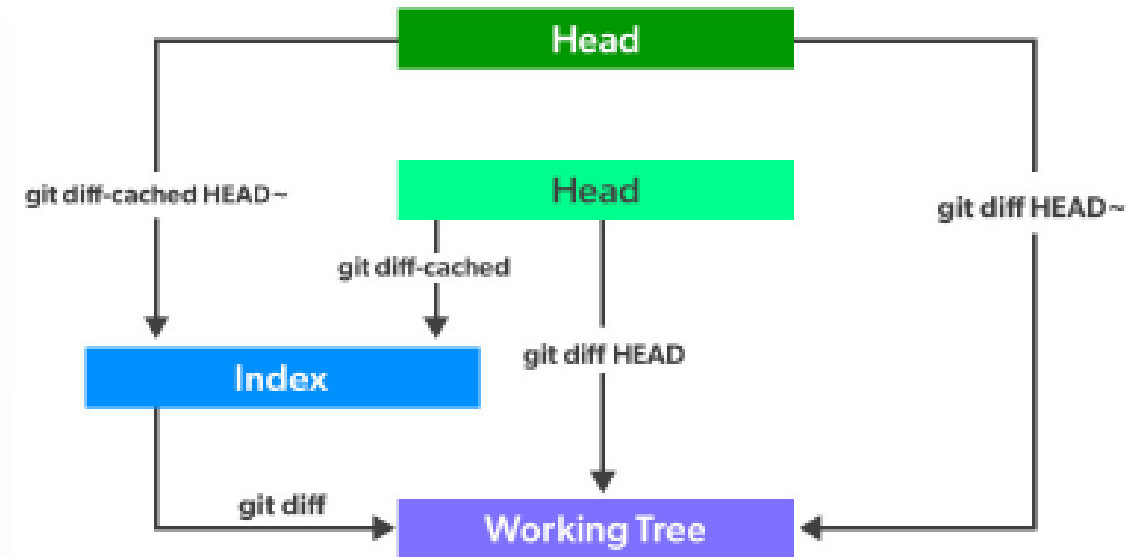
Last Commit

Compare Changes in the Working Area with a Specific Commit

`git diff <commit_id>`

Compare Two Different Commits

`git diff <branch_or_commit_1> <branch_or_commit_2>`



Understanding Git Terminal Output

How to read symbols and colors in `git diff` and `git status`

```
git diff

$ git diff

diff --git a/app.txt b/app.txt
index 98a1b2c..21f4e5d 100644
--- a/app.txt
+++ b/app.txt
@@ -1,3 +1,3 @@
-PORT=8080
+PORT=9000

APP_NAME=devops-demo

$ git status

Changes to be committed:

  modified:   app.txt
```

- + adds
- - removes
- green is usually staged/added
- red is usually unstaged/removed

Read the output like this



Added line

The line is new in the newer version. Usually shown in green.



Removed line

The line existed before but was deleted or replaced. Usually shown in red.



Change location

Shows where the changed block appears in the old and new file.



No symbol

Context line: unchanged text shown only to help you understand the change.

git status colors **GREEN** = staged / ready to commit
RED = modified or untracked, not staged

Note: Terminal colors can vary depending on your Git configuration, terminal theme, or IDE.

Git Checkout

The git checkout command has three primary uses:

- Switching between versions.
- Switching between areas.
- Branching.

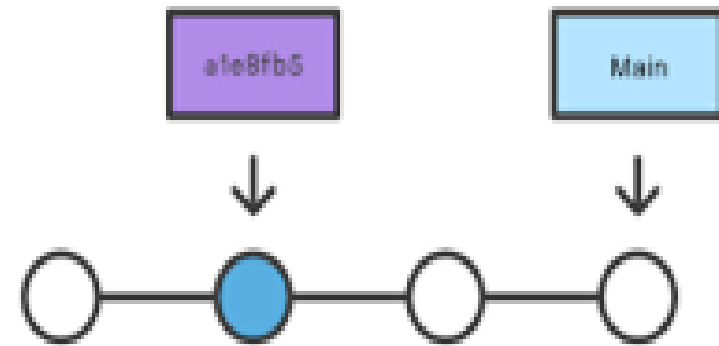
Switch Between Versions (Commits)

- `git checkout <commit_id> .`

Switch Between Areas (Staging and Working Directory)

- `git checkout -- <file_name>`

Checking out a previous commit



Git Branch

Create a New Branch

```
git branch <new-branch-name>
```

Switch from Current Branch to New Branch

```
git checkout <new-branch-name>
```

List All Branches

```
git branch
```

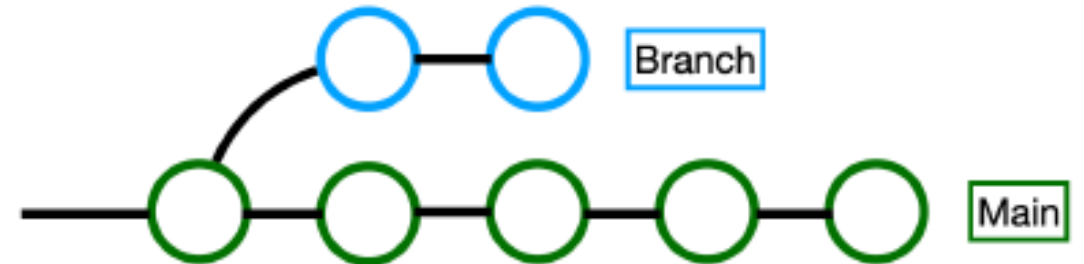
Delete a Branch

```
git branch -d <branch-name>
```

View Remote Branches

```
git branch -r
```

Not : We will revisit this command in the context of GitHub..



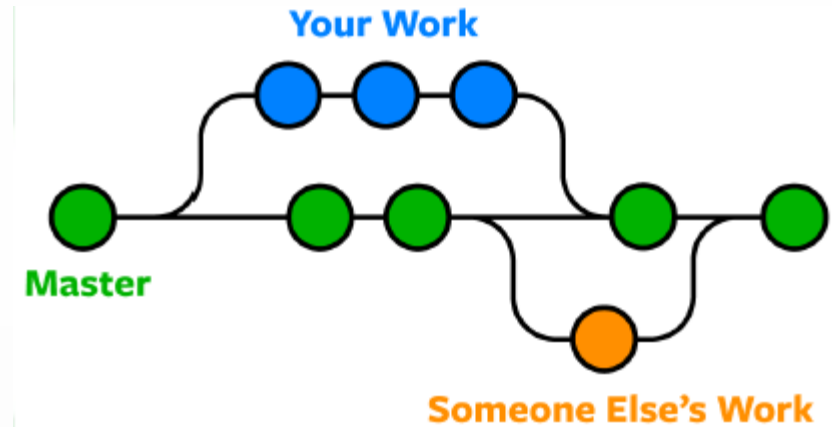
Git Merge

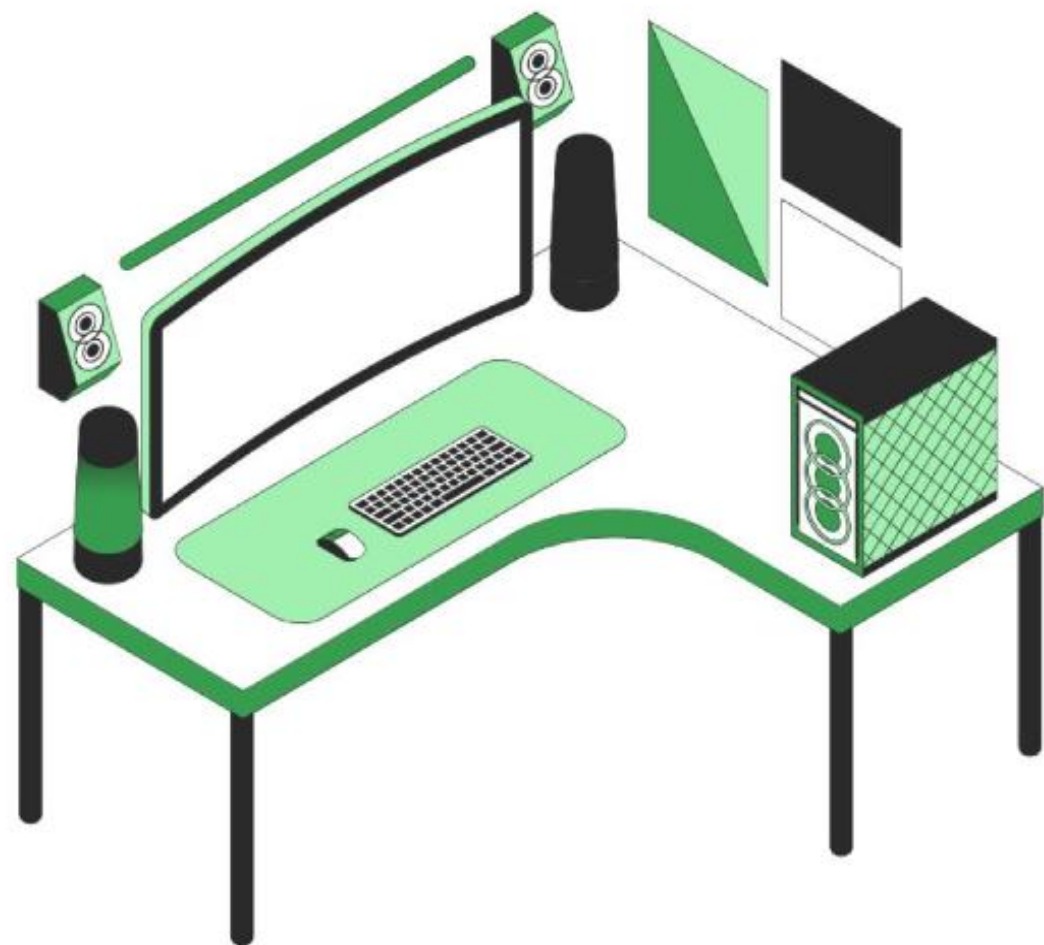
Switch to Main Branch

`git checkout main`

Merge a New Branch into the Main Branch

`git merge <branch-name>`





Do you have any questions?

Send it to us! We hope you learned something new.